

Recruitment & Placement Database Management System

MySQL

A practical, beginner-friendly project for a
Database Intern portfolio

JOINS

CTEs

WINDOW FUNCTIONS

VIEWS

INDEXES

DATA QUALITY

Build a small system that answers real questions

Recruiting teams need reliable answers about candidates, open roles, applications, placements, and invoices. This project models that workflow in MySQL and uses SQL to turn the records into clear operational reports.

Project objective

Design a normalized relational database, load fictional sample records, and write reusable, checked queries that help a recruiting team track its pipeline from job opening to placement and billing.

What the finished portfolio should show

- A six-table schema with primary keys, foreign keys, sensible data types, and clear relationships.
- Readable reports using joins, aggregates, subqueries, CTEs, and window functions.
- Reusable database objects: a view and a stored procedure.
- Basic performance thinking with indexes and EXPLAIN.
- Data-quality checks and a short README that explains how to run the project.

How it matches the internship

Role requirement	Evidence in this project
SQL query writing	Queries for pipeline, hiring, and client reports
Database structure	Six related tables with PKs, FKs, and normalization
Reusable work	View and stored procedure for repeatable reporting
Query performance	Indexes plus before/after EXPLAIN review
Care with data	Duplicate, missing, and inconsistent-value checks

Portfolio honesty

Use fictional data only. First build and run the SQL files yourself; then describe the project as completed on your resume. This guide is a build plan and reference, not proof that the database has already been implemented.

Recommended setup

MySQL 8.0.16 or later, MySQL Workbench or the MySQL command line, and GitHub. Keep the first version small: about 20-40 fictional candidates, 4-6 clients, 8-12 jobs, and enough applications to make reports interesting.

Plan the data before writing reports

The model follows a simple recruiting flow. Applications connect candidates to jobs; placements record a successful application; billing records an invoice for a placement.

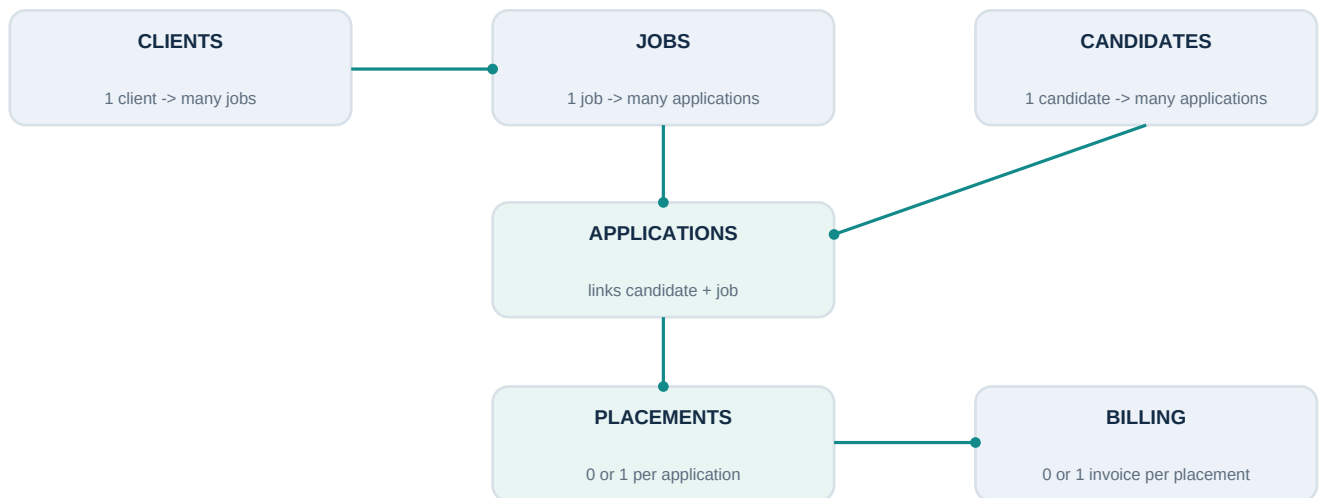


Table guide

Table	Purpose	Key relationship
candidates	One row per person	candidate_id is the primary key
clients	Companies hiring through the agency	One client can have many jobs
jobs	Roles opened by a client	client_id references clients
applications	A candidate's application to a job	candidate_id + job_id are foreign keys
placements	Successful application and joining details	application_id is unique and references applications
billing	Invoice and payment tracking	placement_id is unique and references placements

Normalization in plain language

Store each fact once. Client details live in clients, job details in jobs, and candidate details in candidates. The applications table handles the many-to-many relationship between candidates and jobs. A list of multiple skills would be better modeled with skills and candidate_skills tables in a later version.

Sample schema: core recruiting tables

This runnable starter schema uses InnoDB so foreign keys are enforced. Create the database first, then run the table statements in order.

01_CREATE_TABLES.SQL - PART 1

```
CREATE DATABASE recruitment_placement;
USE recruitment_placement;

CREATE TABLE clients (
  client_id INT AUTO_INCREMENT PRIMARY KEY,
  client_name VARCHAR(120) NOT NULL,
  industry VARCHAR(80),
  city VARCHAR(80),
  created_at DATE NOT NULL
) ENGINE=InnoDB;

CREATE TABLE candidates (
  candidate_id INT AUTO_INCREMENT PRIMARY KEY,
  full_name VARCHAR(120) NOT NULL,
  email VARCHAR(160) NOT NULL UNIQUE,
  phone VARCHAR(25),
  city VARCHAR(80),
  qualification VARCHAR(80),
  experience_years DECIMAL(4,1) NOT NULL DEFAULT 0,
  registration_date DATE NOT NULL,
  CHECK (experience_years >= 0)
) ENGINE=InnoDB;

CREATE TABLE jobs (
  job_id INT AUTO_INCREMENT PRIMARY KEY,
  client_id INT NOT NULL,
  job_title VARCHAR(120) NOT NULL,
  required_skill VARCHAR(80),
  salary_min DECIMAL(10,2),
  salary_max DECIMAL(10,2),
  job_status VARCHAR(20) NOT NULL DEFAULT 'Open',
  FOREIGN KEY (client_id) REFERENCES clients(client_id),
  CHECK (salary_min IS NULL OR salary_max IS NULL OR salary_min <= salary_max)
) ENGINE=InnoDB;
```

Why these keys matter

The primary key identifies one row. A foreign key prevents a job from pointing to a client that does not exist. UNIQUE email prevents duplicate candidate email addresses. CHECK constraints require MySQL 8.0.16 or later.

Finish the schema, then ask useful questions

01_CREATE_TABLES.SQL - PART 2

```
CREATE TABLE applications (
  application_id INT AUTO_INCREMENT PRIMARY KEY,
  candidate_id INT NOT NULL,
  job_id INT NOT NULL,
  application_date DATE NOT NULL,
  application_status VARCHAR(24) NOT NULL DEFAULT 'Applied',
  FOREIGN KEY (candidate_id) REFERENCES candidates(candidate_id),
  FOREIGN KEY (job_id) REFERENCES jobs(job_id),
  UNIQUE (candidate_id, job_id)
) ENGINE=InnoDB;

CREATE TABLE placements (
  placement_id INT AUTO_INCREMENT PRIMARY KEY,
  application_id INT NOT NULL UNIQUE,
  placement_date DATE NOT NULL,
  joining_date DATE,
  placement_status VARCHAR(24) NOT NULL DEFAULT 'Placed',
  salary DECIMAL(10,2),
  FOREIGN KEY (application_id) REFERENCES applications(application_id)
) ENGINE=InnoDB;

CREATE TABLE billing (
  billing_id INT AUTO_INCREMENT PRIMARY KEY,
  placement_id INT NOT NULL UNIQUE,
  invoice_date DATE NOT NULL,
  invoice_amount DECIMAL(10,2) NOT NULL,
  payment_status VARCHAR(20) NOT NULL DEFAULT 'Pending',
  FOREIGN KEY (placement_id) REFERENCES placements(placement_id)
) ENGINE=InnoDB;
```

Begin with a pipeline report

JOIN - CANDIDATE APPLICATION PIPELINE

```
SELECT c.full_name, j.job_title, cl.client_name,
       a.application_status, a.application_date
FROM applications AS a
JOIN candidates AS c ON c.candidate_id = a.candidate_id
JOIN jobs AS j ON j.job_id = a.job_id
JOIN clients AS cl ON cl.client_id = j.client_id
ORDER BY a.application_date DESC;
```

A JOIN combines matching rows using related key values. Add GROUP BY with COUNT, SUM, or AVG when the question asks for a summary, such as placements per client or applications per job.

Show more than basic SELECT queries

Use each technique to answer a specific recruiting question. Keep aliases short and make output columns easy for a reporting team to understand.

AGGREGATION - PLACEMENTS BY CLIENT

```
SELECT cl.client_name,  
       COUNT(p.placement_id) AS placements_made  
FROM clients AS cl  
JOIN jobs AS j ON j.client_id = cl.client_id  
JOIN applications AS a ON a.job_id = j.job_id  
JOIN placements AS p ON p.application_id = a.application_id  
GROUP BY cl.client_id, cl.client_name  
HAVING COUNT(p.placement_id) >= 1  
ORDER BY placements_made DESC;
```

SUBQUERY - CANDIDATES WITH 2 OR MORE APPLICATIONS

```
SELECT c.candidate_id, c.full_name  
FROM candidates AS c  
WHERE c.candidate_id IN (  
  SELECT a.candidate_id  
  FROM applications AS a  
  GROUP BY a.candidate_id  
  HAVING COUNT(*) >= 2  
);
```

CTE - MONTHLY PLACEMENT SUMMARY

```
WITH monthly_placements AS (  
  SELECT DATE_FORMAT(p.joining_date, '%Y-%m') AS month,  
         COUNT(*) AS placements_made  
  FROM placements AS p  
  WHERE p.joining_date IS NOT NULL  
  GROUP BY DATE_FORMAT(p.joining_date, '%Y-%m')  
)  
SELECT month, placements_made  
FROM monthly_placements  
ORDER BY month;
```

Know which tool fits

Use a subquery when one query depends on a result from another. Use a CTE to name a temporary result and make a longer query easier to read. GROUP BY summarizes rows; HAVING filters those grouped results.

Rank results and reuse common logic

WINDOW FUNCTION - LATEST APPLICATION PER CANDIDATE

```
WITH ranked_apps AS (
  SELECT a.candidate_id, a.job_id, a.application_date,
         a.application_status,
         ROW_NUMBER() OVER (
           PARTITION BY a.candidate_id
           ORDER BY a.application_date DESC, a.application_id DESC
         ) AS row_num
  FROM applications AS a
)
SELECT c.full_name, j.job_title, r.application_status,
       r.application_date
FROM ranked_apps AS r
JOIN candidates AS c ON c.candidate_id = r.candidate_id
JOIN jobs AS j ON j.job_id = r.job_id
WHERE r.row_num = 1;
```

ROW_NUMBER assigns an ordered number inside each candidate's group. RANK and DENSE_RANK are useful when ties should share a rank. Unlike GROUP BY, window functions keep the individual rows available.

VIEW - REUSABLE OPEN-JOB LISTING

```
CREATE VIEW open_jobs AS
SELECT j.job_id, j.job_title, j.required_skill,
       j.salary_min, j.salary_max, cl.client_name, cl.city
FROM jobs AS j
JOIN clients AS cl ON cl.client_id = j.client_id
WHERE j.job_status = 'Open';

SELECT * FROM open_jobs ORDER BY client_name, job_title;
```

STORED PROCEDURE - JOBS FOR ONE CLIENT

```
DELIMITER //
CREATE PROCEDURE get_client_jobs(IN p_client_id INT)
BEGIN
  SELECT job_id, job_title, job_status, salary_min, salary_max
  FROM jobs
  WHERE client_id = p_client_id
  ORDER BY job_title;
END //
DELIMITER ;

CALL get_client_jobs(1);
```

Repeatable logic

A view stores a query definition so users can select from it like a table. A stored procedure groups SQL steps behind a CALL. Explain the purpose and inputs of each one in your README.

Check the results and keep queries responsive

A query that runs is not automatically correct. Validate the data and compare important totals before sharing a report.

Data-quality checks

12_DATA_QUALITY_CHECKS.SQL

```
-- Duplicate email check (the UNIQUE key should block new duplicates)
SELECT email, COUNT(*) AS row_count
FROM candidates
GROUP BY email
HAVING COUNT(*) > 1;

-- Missing values worth reviewing
SELECT candidate_id, full_name
FROM candidates
WHERE full_name IS NULL OR TRIM(full_name) = ''
   OR email IS NULL OR TRIM(email) = '';

-- Inconsistent salary range from imported source data
SELECT job_id, salary_min, salary_max
FROM jobs
WHERE salary_min > salary_max;
```

Indexes and EXPLAIN

11_INDEXES.SQL - EXAMPLE COMPOSITE INDEX

```
CREATE INDEX idx_applications_job_status
ON applications (job_id, application_status);

EXPLAIN
SELECT application_id, candidate_id, application_date
FROM applications
WHERE job_id = 3 AND application_status = 'Interview';
```

Index with a reason

An index can speed up filters and joins on frequently used columns, but it uses storage and can slow inserts or updates. Check the plan with EXPLAIN before and after, and write down what changed. Avoid adding indexes to every column.

Simple validation routine

- Count rows in each table after loading sample data.
- Check that every placement points to an application and every job points to a client.
- Manually verify two or three report rows against the source records.
- Confirm totals make sense; for example, placements cannot exceed applications.

Turn the database into a GitHub project

Build in small steps so a reviewer can clone the repository and reproduce your results.

Step	What to do	What to save
1. Plan	Draw the six-table relationship model; decide keys and columns.	README overview or model image
2. Create	Create the database and tables in dependency order.	01_create_database.sql, 02_create_tables.sql
3. Seed	Insert fictional clients, candidates, jobs, applications, placements, invoices.	03_insert_data.sql
4. Query	Add examples from basic SELECT through window functions.	04-08 query files
5. Reuse	Create a reporting view and a stored procedure.	09_views.sql, 10_stored_procedures.sql
6. Improve	Check a useful index with EXPLAIN; run quality checks.	11_indexes.sql, 12_data_quality_checks.sql
7. Document	Add setup steps, example questions, screenshots, and limitations.	README.md and screenshots/

SUGGESTED GITHUB FOLDER STRUCTURE

```
recruitment-placement-database/
|-- README.md
|-- database/
|   |-- 01_create_database.sql
|   |-- 02_create_tables.sql
|   |-- 03_insert_data.sql
|   |-- 04_basic_queries.sql
|   |-- 05_joins.sql
|   |-- 06_aggregations.sql
|   |-- 07_subqueries_cte.sql
|   |-- 08_window_functions.sql
|   |-- 09_views.sql
|   |-- 10_stored_procedures.sql
|   |-- 11_indexes.sql
|   |-- 12_data_quality_checks.sql
|-- dataset/      (optional CSV exports of fictional data)
|-- screenshots/  (schema and selected query results)
```

README checklist

State the objective, MySQL version, setup order, table relationships, key queries, sample results, index experiment, data-quality checks, and known limitations. Include screenshots only after running the queries yourself.

Explain the work clearly in an interview

Use this as a structure, then replace the bracketed details with what you actually built and measured:

60-second project explanation

I built a Recruitment and Placement Database in MySQL to model how candidates move through job applications, placements, and billing. I designed six related tables with primary and foreign keys, then wrote reports using joins, aggregations, subqueries, CTEs, and window functions. I also created a view and a stored procedure, checked sample data for duplicates and invalid values, and used EXPLAIN to review an index on [columns]. The project uses fictional data, and I documented the setup and query results in GitHub.

Resume bullet - customize after implementation

Built a MySQL recruitment and placement database with six normalized tables; wrote reporting queries using joins, CTEs, subqueries, and window functions; added reusable views, a stored procedure, data-quality checks, and an EXPLAIN-based index review.

Questions to practice

- Why does applications connect candidates and jobs?
- When would you use HAVING instead of WHERE?
- How is a CTE different from a subquery in the way you used it?
- What did EXPLAIN show before and after your index?
- How did you verify that your placement count was correct?

Before you publish

- Run every SQL file from a clean database in the order shown.
- Replace all example records with fictional data you created.
- Check output values and screenshots; never claim an unmeasured performance improvement.
- Add your own name and repository URL to the README, not to this guide.

You do not need to know every advanced topic on day one. A clear schema, accurate results, and an honest explanation of your choices make this a strong beginner portfolio project.